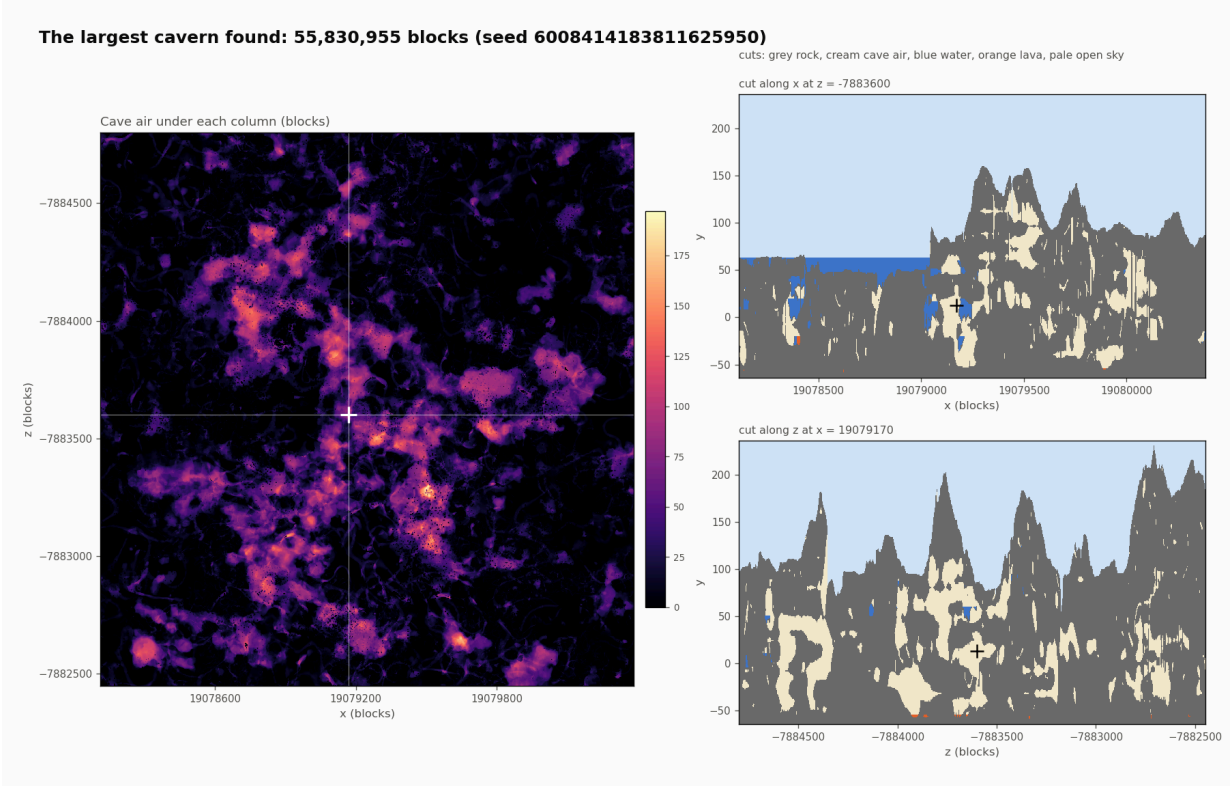
The largest cave by volume: 55.8 million blocks

The task: find the largest cave in Minecraft 26.3 by volume, with a measure worth optimising and a search fast enough to push it. The best cave so far is in seed **6008414183811625950** at `/tp 19079170 13 -7883600`: **55,830,955 blocks** of connected cavern, 2,042 by 2,130 blocks across and 238 tall, checked block for block against the game's own terrain code. The claimed record at the start was about 24M. Three rentals did it, one after the other: an EPYC with an RTX A2000 ($0.16 an hour, $2.00 in all), an EPYC with an RTX 4080 Super ($0.30 an hour, $1.51), and the same machine again for $1.00 to test faster code. In its first four hours the search found no cave of 40M+; at the end it found four a minute. Times are UTC, 5 and 6 October 2026.



The record cavern. Left: cave air under each column. Right: two vertical cuts through the /tp point.

1. The measure

A cave here is a **cavern**: underground air (air with something solid above it, no water or lava) within 3 blocks of a connected set of *core* blocks, a core block being the centre of a 7×7×7 cube of underground air. Two chambers count as one cavern only if a passage at least 7 blocks wide joins them, so thin tunnels do not chain everything into one. Only the noise terrain and its aquifers count (no carvers, features or structures), which is what the world generator fixes before anything else and what can be computed from the seed alone.

Everything is exact in the end: `native/cavemeasure` is a C++ port of the game's density function and aquifer that floods the cavern on 320-bit columns, and the game's jar itself (`measure.sh`) confirmed each record with the same volume and bounding box.

2. The search, stage by stage

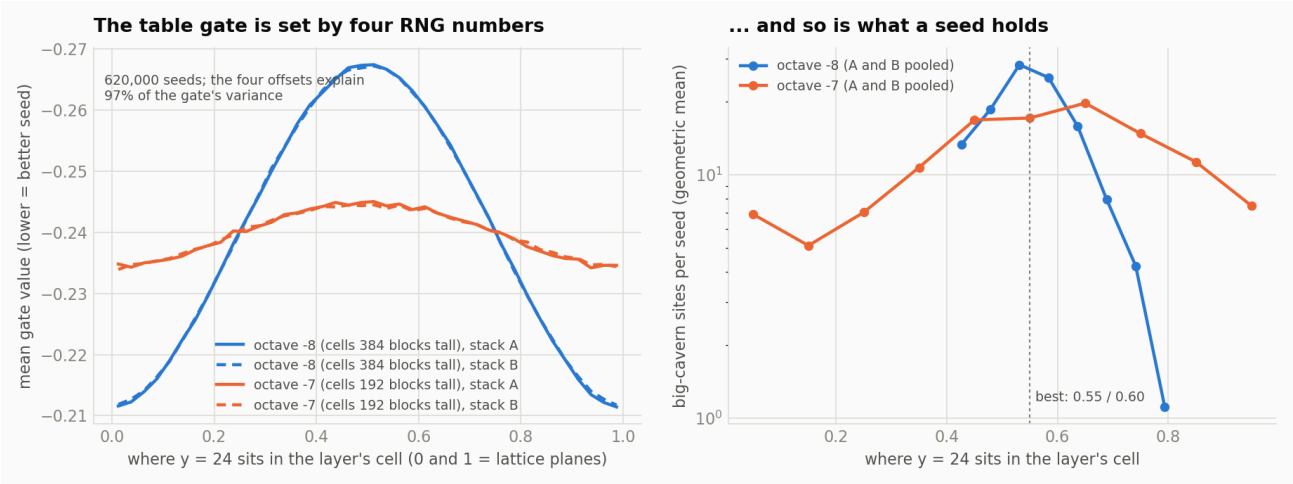
A cheese cave appears where the `cave_cheese` noise is low. That noise is two stacks of Perlin layers, A and B, at frequencies in the ratio 337:331, and each stack repeats every 65,536 blocks (64,364 for B). So a low spot of A and a low spot of B meet at two or three places per axis in the world, which the Chinese remainder theorem gives directly: nobody has to walk the world to find them. That is the **pair trick**, and every later stage is a way of asking "which seeds, and which pairs, are worth the next stage?" more cheaply.

stage	what it decides	where	cost
phase scan	are the y offsets of the four lowest cheese layers where the noise swings hardest at cave height, and those of the 12 lowest terrain layers where terrain is tallest? (RNG steps only, no noise)	GPU	120 billion seeds/s
coarse looks	the lowest 1,280-block window mean of each stack, from the permutation tables: octave -8 first, then both octaves, then in full	GPU and CPU threads	0.2 µs on the card, 8 µs on a thread; then 9 µs and 70 µs for the few left
table gate, first look	window means on a 256-block grid and pair counts, for the 1.2% past the coarse looks	GPU	7,000 seeds/s
table gate, full	the same on the full grid, and the connected low-noise volume (percolation term)	CPU	16 ms
scanner	the 200,000 best pairs, the terrain at each of their places, clusters on fine tables	GPU tables, formula and terrain; CPU clusters	1.1 s
formula	the cheese formula on a 16-block grid around the best sites: largest cluster F and open volume O	GPU samples, CPU clusters	5 ms a site
exact measure	the real cavern, block by block	GPU densities, CPU blocks and flood	2 to 5 s a cavern

(Costs on the last box. About one seed in four billion scanned reaches the scanner.)

3. The ideas, in the order they came

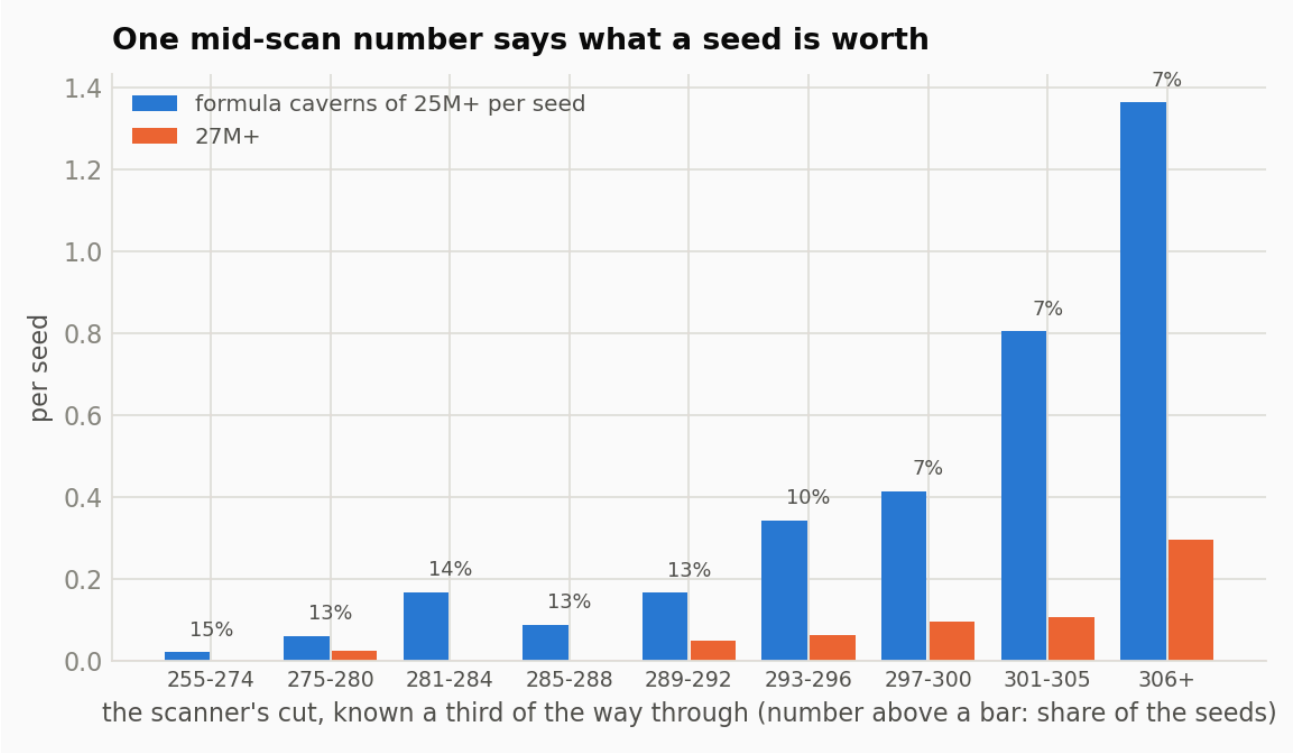
Phases from the RNG (02:41). A Perlin layer is weakest on its lattice planes and strongest half way between them. The two lowest cheese octaves have cells 384 and 192 blocks tall, so where their planes fall in y decides how hard the noise can swing where caves live, and that is one random number per layer. Four numbers predict the table gate of a seed with r = 0.986. The same holds for the terrain layers. This was worth a factor 8.



Left: the gate value against the y phase of the four lowest layers. Right: what a seed holds against the same.

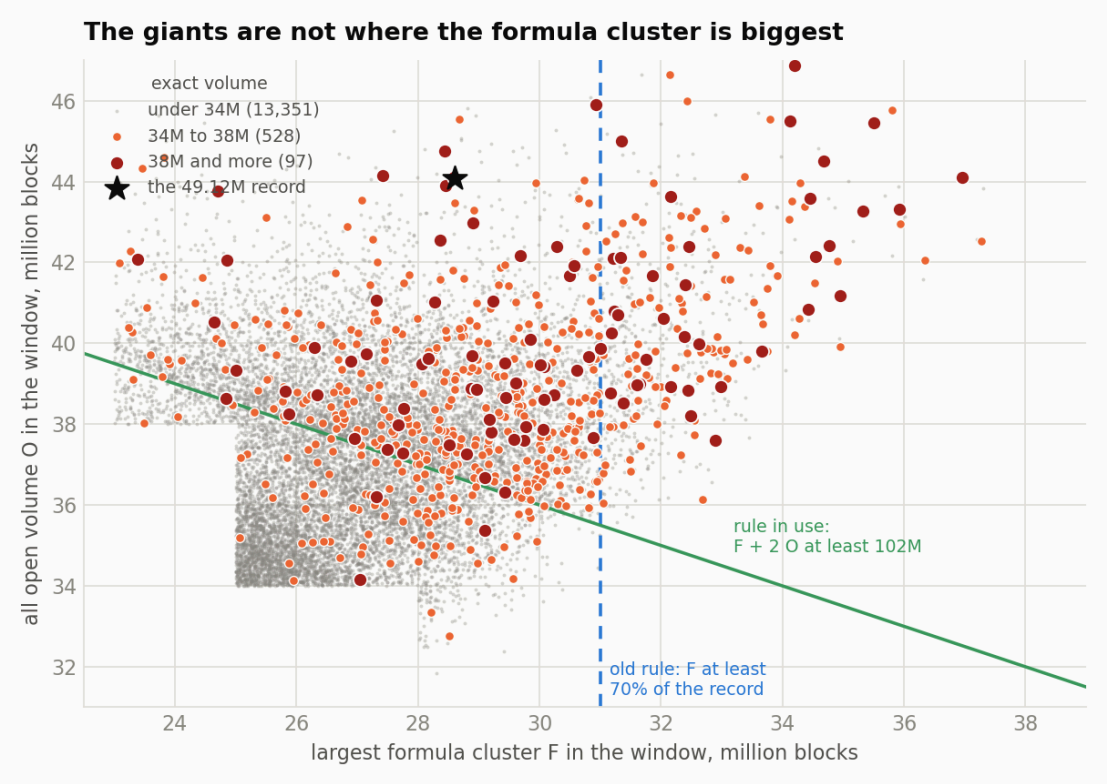
The table gate without noise samples (03:35). The mean of a Perlin layer over a box is a closed form in the gradients at the lattice corners, so the window means come straight from the permutation tables: 8 ms a seed instead of 90.

Terrain first, and stop early (04:21). The scanner looks at the terrain above every place of every pair before anything expensive, and one number a third of the way through (how many open points the 1,400th best place has under its surface) says what the seed is worth. Seeds under a threshold stop there.



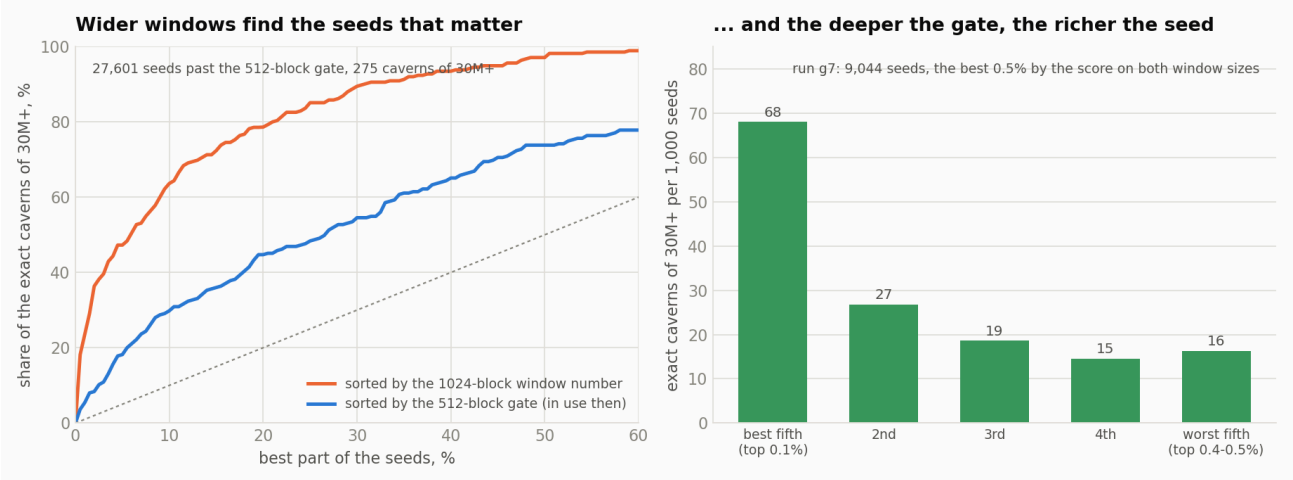
Formula caverns per seed against the scanner's mid-scan number.

The exact measure was looking in the wrong place (05:24). The giants are 1,400 to 2,400 blocks long, longer than the formula's 1,536-block window, and they are often several formula clusters that join in the real terrain. Measuring only the sites with the biggest cluster caught 2 of the 5 known caves over 38M. Measuring by cluster *and* open volume tripled the giants found; re-measuring sites already on disk turned up a 40.7M cave that had been skipped.



Every cavern measured under the rule on both numbers. The old rule kept what is right of the dashed line.

Wider windows in the gate (05:40, 06:06). Since the giants are long, the seed gate should average the noise over more than 512 blocks. Windows of 1,280 blocks predict a seed's big caverns far better, and the 512-block number still counts beside it, more so the bigger the cavern. The score in use is the sum of both and the terrain score.

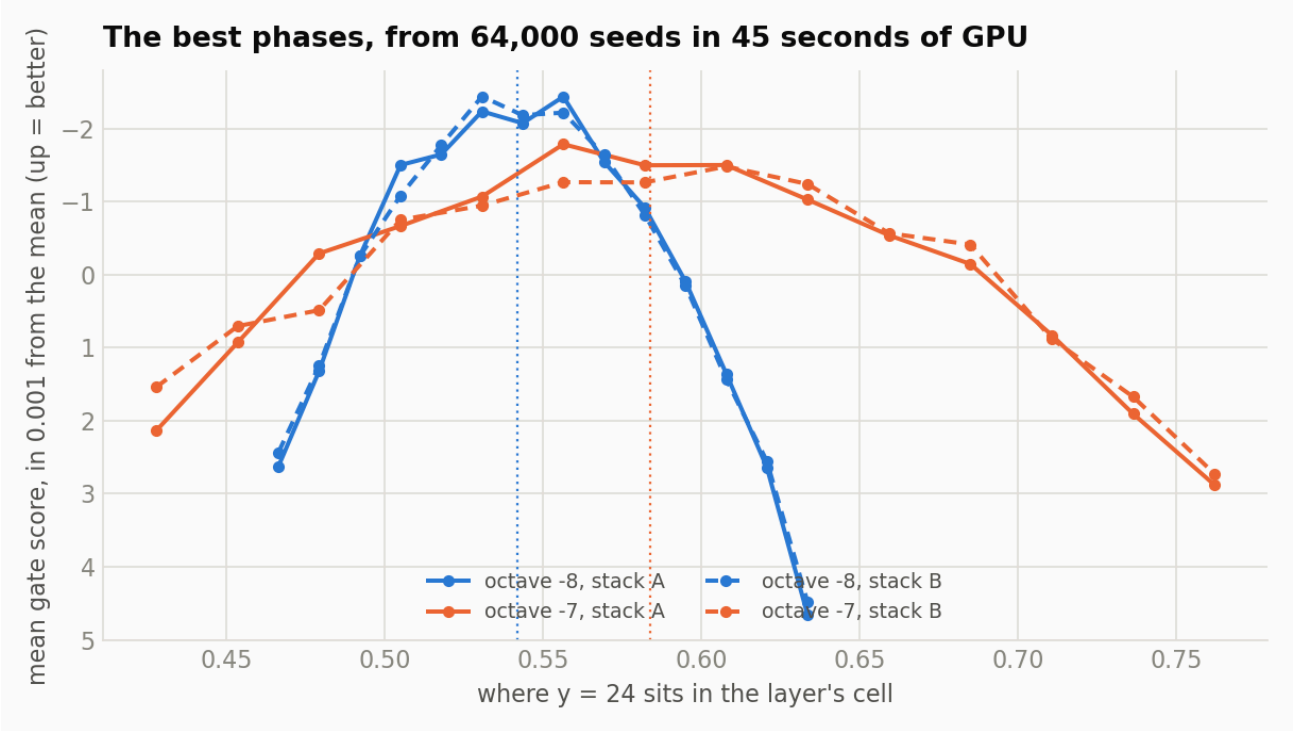


Left: share of the big caverns in the best part of the seeds, by each number. Right: yield by fifth of the score.

The GPU (06:48, 07:04). The box's GPU had been idle. The phase scan is pure 64-bit arithmetic: 18 billion seeds a second on the card against 0.1 billion per CPU thread. The table gate went there too, as a first look on a coarse grid that follows the full score with r = 0.983: one block of 256 threads per seed, with every intermediate in shared memory (the first version kept them in global memory and was no faster than the CPU).

The gate now looks at 13,000 seeds a second where twelve CPU threads managed 600, and keeps the best 0.07% instead of the best 0.5%. The two GPU steps together were worth a factor 2.5 in caves of 30M+ per hour and 3.5 at 36M+.

The GPU also made measurements cheap that had been guesses. In 45 seconds it gave the score of 64,000 seeds over phase windows three times as wide, which moved the phase centres from 0.550 and 0.595 to 0.542 and 0.584 and widened the windows to fit the card's time split.



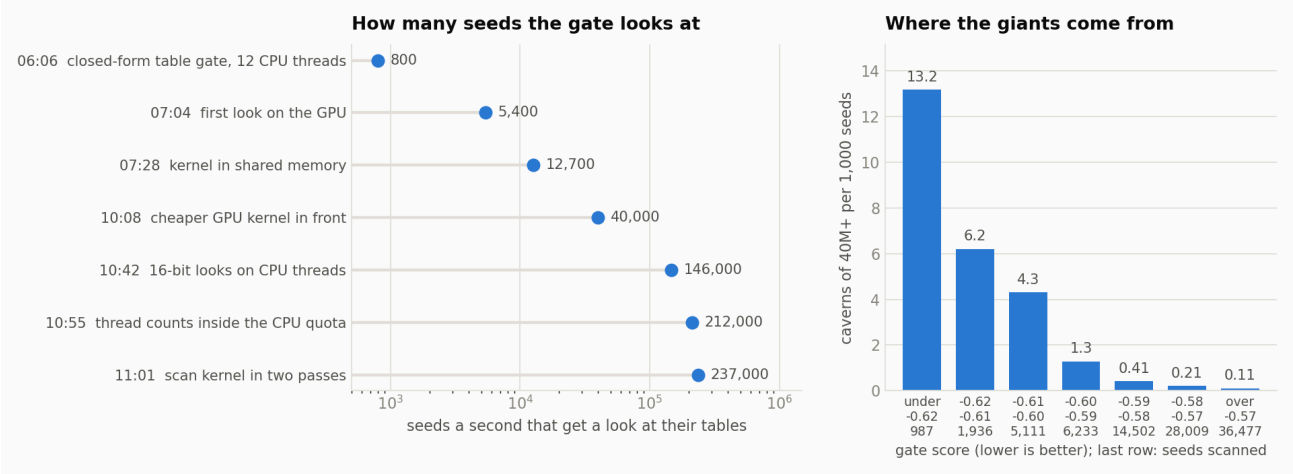
The gate score against the y phase of the four layers, both stacks. Dotted: the fitted optima.

Percolation (08:20). The biggest caverns look like percolation clusters: many blobs joined into one sponge, and that is what they are. A giant forms where the *smooth* part of the noise (the two lowest octaves, both stacks added) stays under a critical level over a connected region about 1.2 km across, whatever its shape; inside such a region nearly all the open space hangs together. A complete search over all 65,536 offsets between the two stacks for such regions, with the terrain at every world place each offset produces, ranks the 49.1M record as the **number-one place of its seed**; over 40 seeds with known giants a third of the giants are their seed's best place, two thirds are in its best 10 and 96% in its best 150.

That search found no giant the scanner had missed (of 43 top places that pass the exact rule, 38 had been measured and the other 5 came out at 19 to 28M), so the scanner stays. The idea went into the gate as a cheaper term (the connected volume over the best alignments of each stack's lowest spots, 5 ms a seed), which sorts already-gated seeds better offline but changed nothing measurable live: the workers already take the best seeds first, and what limits the search now is how many seeds the GPU can look at.

The complete version is the natural next gate. The value of a seed's best place separates the seeds that hold a giant from the rest more sharply than anything else measured here: 0.95 or more for 88% of the giant seeds and for 15% of other gated seeds. But the brute-force search takes 55 CPU-seconds a seed, and it only pays when applied to ten times more seeds than the scanner sees now, which needs a sparse algorithm on a bigger GPU.

Where the giants come from (10:00). With 93,000 scanned seeds on record, the answer to “what limits the largest caves?” was in the data: they come from the very top of the gate’s list. The best band of the score holds a hundred times more caverns of 40M+ per seed than the bulk of what the workers were scanning, and the number of seeds falls by 10 for every 0.0225 of score. So the largest caves are a matter of how many seeds the gate can look at, and most of the box was busy with seeds that almost never hold one.



Left: seeds a second the gate looks at, step by step. Right: caverns of 40M+ per 1,000 scanned seeds, by gate score.

The cheap look on a CPU thread (10:42). The GPU’s table kernel was the limit at 12,700 seeds a second, and a cheaper kernel in front of it (octave -8 alone, coarse windows) got 40,000. Then the same look was written for a CPU thread. Two things make it small. A wide window’s mean is carried by the y components of the lattice gradients (the other two only count at the window’s edges), and those are -1, 0 or 1 times a weight, so the whole look fits 16-bit integers: a byte shuffle turns 32 table entries into gradients at once. It takes 18.5 µs a seed on one laptop thread, where the whole card had needed 10.5 µs, and it sorts seeds as well as the full number. A second 16-bit look adds octave -7, the full two-octave number confirms, and 1.2% of the seeds go on to the GPU, which is back to scanning.

The CPU quota (10:55). The container may use 61 threads, and gate plus workers were running a hundred. Linux then freezes every thread of the container for part of each tenth of a second, the one that drives the GPU too, so the card sat idle a third of the time. With thread counts that fit the quota (27 for the gate, 35 single-threaded workers) the scan went from 10 to 14 billion seeds a second with nothing else changed. A scan kernel that tests the first layer for a whole piece of seeds before the other three added 18%: with all four tests in one loop, a GPU warp ran nearly every step for every seed.

Workers for the top only (10:50 to 11:13). With six times more top seeds arriving, the workers were trimmed for them: the exact measure only for sites with F + 2 O of 107M or more (30% of the measurements), fine tables for 1,250 places a seed instead of 5,000 (all 36 known caverns of 42M+ are found either way). The first of these cost more than it looked when it was set: of the 64 caverns of 40M+ found before it, 48 pass it (and 12 of the 14 of 44M+), not the nine in ten a smaller sample had suggested. It bought twice the top seeds per worker, which about evens out; 105M would have kept 54 of the 64. A refit on the 93,000 seeds also gave the terrain score three times its old weight in the ranking.

A bigger card, and the gate on it (17:20). The second box has an RTX 4080 Super beside the same 61 CPU threads. The terrain score went into the scan kernel (the host had been working it out for five million seeds a second) and the first coarse look became a GPU kernel, 0.7 µs a seed. On the same 27 threads the gate looks at 647,000 seeds a second where the CPU-look version manages 368,000.

The scanner on the GPU (17:37 to 19:15). With the gate that fast the workers were the limit, and their time was 61% exact measure, 31% scanner, 8% formula stage. Four pieces of that moved to the card. One process (`gpusrv`) holds the card for the workers and answers them over a socket:

- the scanner's terrain step: 1.5 million places a seed, up to 16 surface estimates each (Perlin octaves and the game's offset spline). A million places come back in 35 ms; the CPU needed over a second. Same decisions.
- the scanner's coarse tables and its formula step, and the formula stage's grids of open samples. Same sites.
- the exact measurer's density functions at the cell corners, which were 62% of a measurement. The generated density code is compiled for the card and gives the CPU's values bit for bit (410,000 compared, none different) at 0.3 µs a column; a CPU thread takes 130 µs.

A worker's time per seed went from 14.4 s to under 5, and the box scans 39,000 seeds an hour instead of 8,700.

What it took to make the kernels fast is worth keeping. The generated functions must stay calls: inlined, the kernel was 10 MB and the driver took a quarter of an hour to load it. A launch is as slow as its slowest thread, so the work is split into short threads (a column's cache first, then one thread per column and level). Double precision runs at a sixty-fourth of single on this card, so the lattice arithmetic is done once per column and once per level instead of per sample: five times faster. And hundreds of small requests are gathered into one launch.

Where the card runs out (19:40 to 22:17). With everything on it, the card was the limit for gate and workers alike, and three more changes went into the CPU side. The exact measurer asked the card for chunks ring by ring as its flood reached them, 119 round trips for a 54M cavern, each waiting in a queue: asking a few rings ahead made that a dozen. Its block loop was vectorised (16 columns of a cell at a time, all five interpolated functions; open cells by height masks; blocks whose twelve candidate aquifer centres hold the same fluid skip the search), which took the cells and aquifers of a 54M cavern from 6 s to 2.7 s on the loaded box, bit for bit the same. And a set number of the gate's threads took coarse looks from the card. The rates moved (seeds scanned a minute 648 to 761, looks a second 458,000 to 479,000) but the curve did not: 581 ± 36 caverns of 40M+ per dollar after, 661 ± 56 before. Faster workers had only taken card time from the gate.

The z kernel, once (third rental, 22:48 on). The owner of this search did not believe the code was at its limit, and asked for another look at it the way a rival's kernels had once shown a threefold gap in a sister project. The look found this. A window mean is a weighted sum of lattice gradients along x, y and z. The gradient at lattice point (x, y, z) is `d[(d[(d[x] + y) & 255] + z) & 255]`, so along z every lattice line reads the *same ring of 256 gradients*, turned by one byte that depends on x and y. The windows' kernel along z can therefore be applied once, to that ring, and a line that is already summed over a window along z is then a plain read of the filtered ring from a turned start. The old code made each of the 261 lines of a layer from 256 gradient lookups per height and filtered each of the 128 rows along z again.

	before	after
coarse look on a CPU thread (16-bit)	32 µs	8.5 µs
full look on a CPU thread (float)	213 µs	38 µs
coarse look on the card	0.65 µs	0.20 µs
first look at the tables on the card, seeds a second (card shared)	37,600	64,700
full gate on a CPU thread (with a faster spot search and pair count)	39 ms	16 ms
gate alone on the card, seeds a second past the scan	652,000	1,015,000 (1,196,000 with 24 threads on looks)

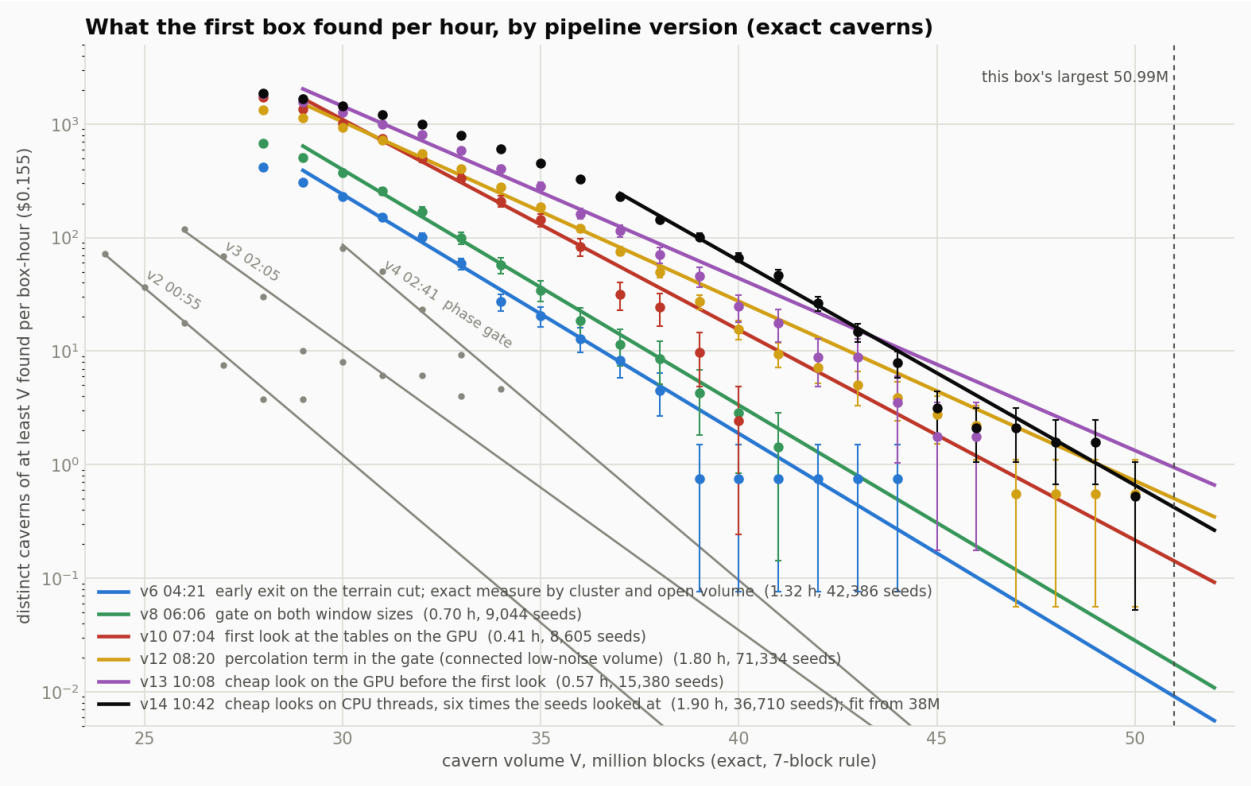
The numbers are the same ones (the float looks to 6e-8; the 16-bit ones with no cut decided differently in 3,000 seeds). Two smaller things came with it. The scan now walks seeds along a chain: the game starts a seed's generator from two mixes, of `lo` and of `lo + GOLDEN`, so for seeds whose `lo` values are `GOLDEN` apart the second mix of one is the first of the next, and a thread makes one mix per seed instead of two (9 to 17% more seeds a second; the seeds now come from anywhere in the 64 bits, and half of them are negative numbers). And the scan kernel must not be compiled for the card's own architecture (`-arch=sm_89`): built that way it runs at a fifth of the speed, while every other kernel is as fast or faster with it. The whole pipeline went from 479,000 looks a second and 761 seeds scanned a minute to 790,000 and 950, and the gate listed 380 top seeds a minute where it listed 233. A cavern of 55.2M came 20 minutes into this build.

Stages that talk to each other (00:35 to 00:50). Two last changes came from asking where a stage still ignores what another knows. In the gate, the threads set aside for CPU looks were half idle, because the other threads took the scan's output for the card first; with the card looking only at what they leave, the gate went from 784,000 to 890,000 looks a second. And the exact rule had ignored the seed's gate score, which is independent of the two formula numbers (correlation -0.03) and still counts at that stage: for a cavern of 46M+, -0.28 ± 0.10 per standard deviation over 74,000 measured sites. The floor on F + 2 O now rises for seeds that only just made the list. On the sites already measured, 74% of the measuring time keeps 49 of the 50 caverns of 46M+ and all 18 of 48M+; on the box the share of seeds that need an exact measure fell from 22% to 16% and the seeds scanned a minute rose from 850 to 908. The coarse looks still ignore the terrain score that the scan has already worked out; a joint cut there is the same idea and is not built. The record of 55.8M was measured two minutes after these went in.

---

4. The rarity curve

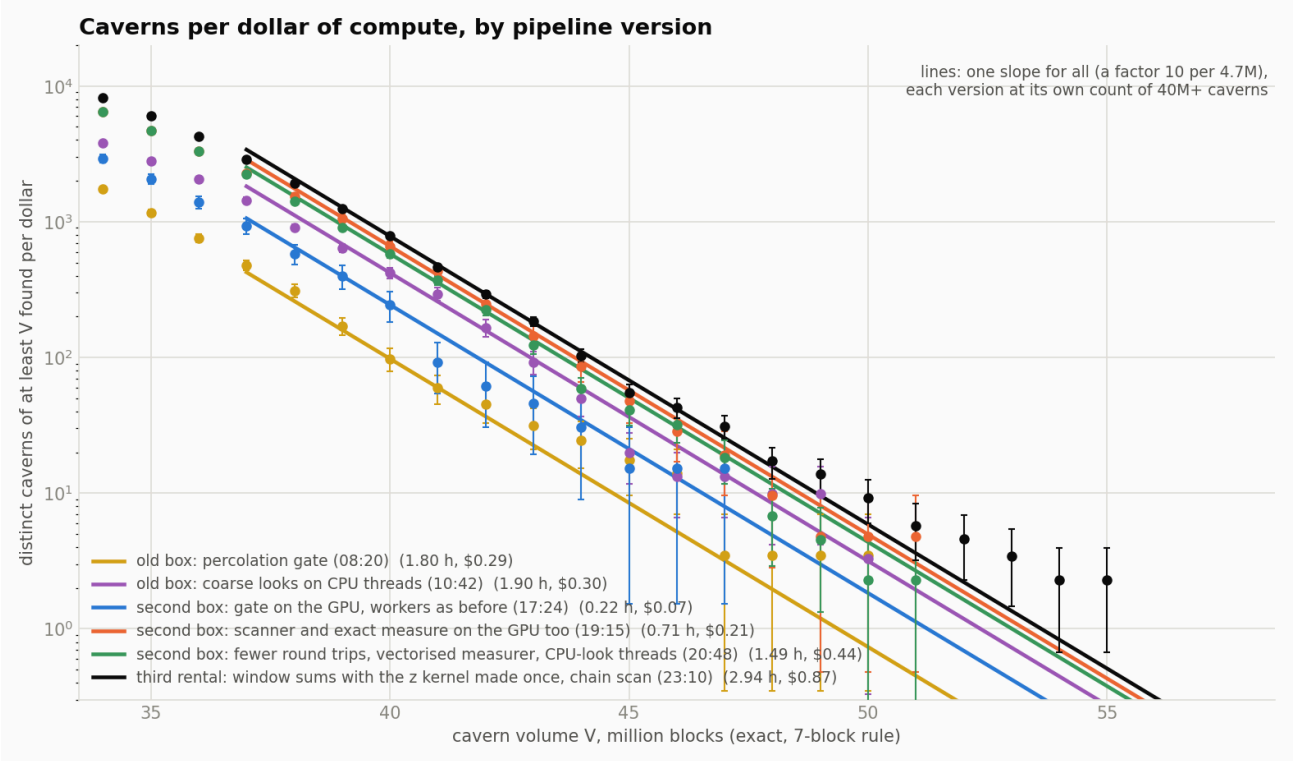
Big caverns thin out exponentially. Every step above raised the curve, and two of them made it flatter: the exact rule on two numbers and the wide gate (a factor 10 for every 3.4 million blocks in the early fits, 6.3 million with the percolation gate). The first box's last version, in 1.9 hours, raised the curve by a factor 3 to 4 between 38M and 42M and by 2 at 44M. At 46M+ it found 4 caverns, as the version before it had in 1.8 hours: the far end is not shown to have moved. The gate did its part (17 times more seeds in the best band of the score per hour, and per scanned seed exactly as many of them inside the old filters as the old gate found); the stricter exact rule took some of it back, and with four caverns on each side the rest is not measurable. It measures a cavern only if F + 2 O reaches 107M, which leaves out most caverns under 38M, so its counts below that are not comparable and its line is fitted from 38M up.



Distinct caverns of at least V found per hour on the first box, by pipeline version, from exact volumes.

caves per box-hour	30M+	34M+	38M+	40M+	42M+	44M+	a factor 10 per
v2 00:55 fast exact measure	1.2	0.08					3.4M
v3 02:05 terrain-first scanner	11	1.1					3.4M
v4 02:41 phase gate	87	5.7					3.4M
v6 04:21 early exit; exact by F and O	231	27	4.5	0.8	0.8	0.8	4.7M
v8 06:06 gate on both window sizes	373	58	8.6	2.9			4.8M
v10 07:04 scan and first look on the GPU	1,021	210	24	2.4			5.4M
v11 07:28 phase windows from the GPU, leaner workers	945	257	47	22	12	5.8	6.4M
v12 08:20 percolation term in the gate	937	277	49	16	7.2	3.9	6.3M
v13 10:08 cheap look on the GPU, wider filters	1,265	404	71	25	8.8	3.5	6.6M
v14 10:42 cheap looks on CPU threads (fit from 38M)			144	67	26	7.9	5.1M

Per dollar. The two cards cost differently, so across them the curve to read is caverns per dollar, and the number it stands for is the largest cavern to expect from a budget: where the fitted rate times the budget is one cavern. The slopes of the single builds scatter within their errors, so one slope is fitted to all of them and each build keeps its own level. A build that finds f times the caverns per dollar moves the expected largest cavern of any budget by the slope times log10 f.



Distinct caverns of at least V found per dollar of rented box, by build. Lines: one slope for all, each build at its own count of 40M+ caverns.

build	hours	cost	38M+	40M+	42M+	44M+	46M+	largest to expect for $1	for $10
box 1, percolation gate	1.80	$0.29	311	98	45	24	14	49.4M ± 0.4	54.1M
box 1, last build (CPU coarse looks)	1.90	$0.30	907	420	166	50	13	52.4M ± 0.2	57.1M
box 2, gate on the GPU	0.22	$0.07	580	244	61	31	15	51.2M ± 0.5	55.9M
box 2, workers on the GPU too	0.71	$0.21	1,542	661	249	86	29	53.3M ± 0.2	58.0M
box 2, last build	1.49	$0.44	1,421	581	225	59	32	53.0M ± 0.1	57.7M
box 2, whole run	4.95	$1.46	1,267	523	199	64	25	52.8M ± 0.1	57.5M
box 3, z kernel made once (all of it)	2.94	$0.87	1,924	783	292	104	43	53.6M ± 0.1	58.3M
box 3, its last 75 minutes	1.29	$0.38	1,983	823	329	121	45	53.7M ± 0.1	58.4M

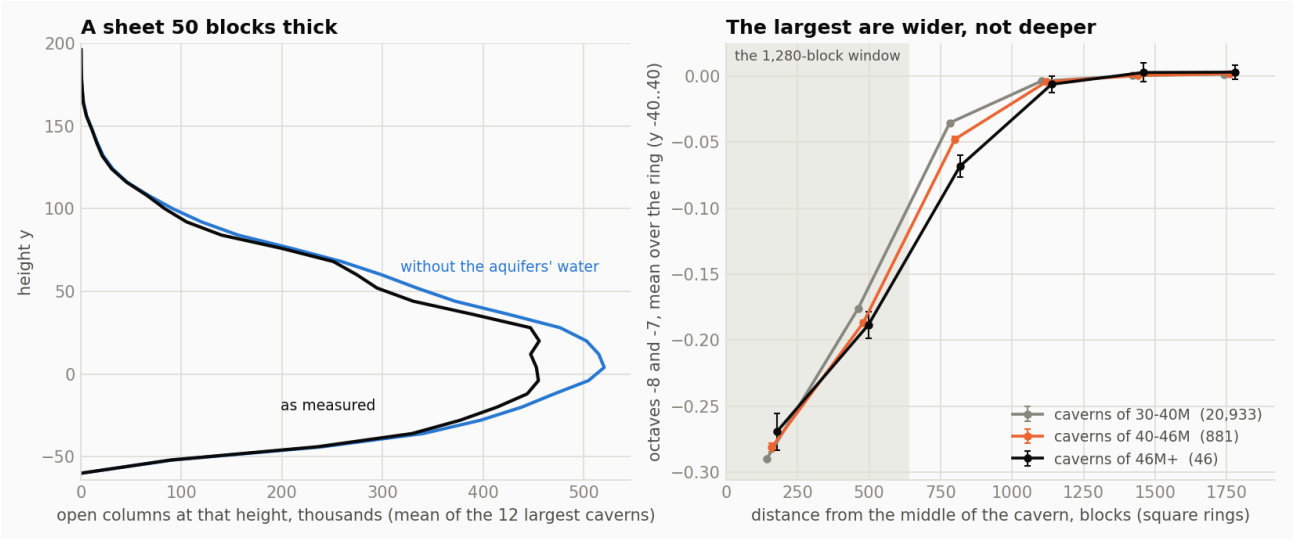
(Distinct caverns of at least that volume per dollar of rented box. Largest to expect: where the fitted rate times the budget is one cavern, with one slope for all builds, a factor 10 per 4.7M ± 0.1M from 1599 caverns of 40M+; the ± is from the build's own count of 40M+ caverns.)

By this measure the second box's builds found 1.4 to 1.6 times the first box's caverns of 40M+ per dollar, and the third rental 1.9 times (783 per dollar over its three hours, 823 in its last 75 minutes): +1.3M on the largest cavern to expect from any budget. The far end moved more than the middle (46M+: 43 per dollar, from 37 caverns, against 25 to 32 on the second box), which is what section 5 expects of a more selective gate.

---

5. What limits a cavern

Asked what limits the size, the data says: the footprint. A cavern of 50M is a sheet, 1.0 to 1.2 million columns of footprint (a quarter of its 1,700 by 2,200 bounding box) times 44 to 51 blocks of open height, with 65% of its volume between y -40 and 40 and 9% above y 80. Size classes differ in footprint and not in thickness, which falls slightly with size.



Left: open columns by height, mean of the 12 largest caverns, as measured and with aquifer water removed. Right: the two lowest octaves around caverns of three size classes (bars: two standard errors).

Re-measuring 24 caverns under changed rules gives the budget. Without the aquifers' water they would be 9% larger (4 to 21%; the cavern of 54.6M would be 64.8M); lava costs 0.2%; and if a passage needed to be 5 blocks wide instead of 7 to join two chambers, 13% larger (7 to 34%). The thickness is capped everywhere: the game adds $4 \times$ `cave_layer`^{2} to the cheese noise, which leaves about 40% of the heights open wherever the cheese is low, and that term varies little between places (0.405 ± 0.038 on average; caverns sit at 0.365, with no trend by size).

What sets the largest apart is on the right of the figure: at the middle they are no deeper than the rest, but 640 to 960 blocks out, beyond the 1,280-block window the gate ranks by, their noise is 0.9 standard deviations lower. A window of 1,536 blocks carries that into the gate (it adds to the score at z = -3.5 to -4.7), but it raises the curve by a few percent and does not flatten it: re-ranking the 169,000 scanned seeds with it puts 38 instead of 32 caverns of 44M+ in the best tenth, with the same slope (a factor 10 per 5.25M against 5.23M). What does flatten the curve is selectivity on the score itself: the slope is a factor 10 per 4.71M over all scanned seeds, 4.99M in the best quarter and 5.23M ± 0.34 in the best tenth. That is what a faster gate buys.

No other number tried predicts the size among candidates: the next octave down (its window means are six times smaller), the aquifer noises' height phases (0.08M of effect), and the formula stage on windows of 2,560 or 3,072 blocks (correlation with the exact volume 0.2 to 0.27 among measured sites, as at 1,536: to keep nine in ten of the 44M+ caverns, three quarters of the sites must be measured whatever the window). What decides between 40M and 55M at a good site is whether chambers join at the 7-block scale, and only the exact measure sees that.

---

6. What was tested and left alone

Gate depth between the best 0.2% and 0.5% on the CPU, the scanner's stop threshold over a wide range, other height ranges for the window means, the formula at 8-block steps or in a 2,304-block window as a predictor, tighter phase windows, pairs of neighbouring formula sites, and a scanner that goes straight to a seed's best places all came out flat or worse. A real block lattice in place of the formula, window-level count proxies and more formula octaves were dead ends.

On the second box: a quick estimate of a cavern from densities on a lattice twice as wide, to decide which caverns get the exact measure (three times faster, but a cut on it keeps 8 of 13 caverns of 40M+: about 0.7 times the caverns per dollar); a cap on the gate's share of the card, and CPU threads taking the looks the GPU does not get to (both gave the workers less than they cost in looks).

Late in the run, with the looks on the CPU: half the pairs in the scanner (loses 1 of 36 known giants for a tenth of the worker time), a stricter cut before the full gate, other weights for the percolation volume (flat between 0.09 and 0.22), wider phase and terrain filters (the seeds they add are worth three quarters of the average and cost the same CPU), and more expensive hardware (nothing on offer beats this box per dollar now that the looks run on CPU threads).

On the last two rentals: a looser terrain cut in the scan (-0.38 for -0.45: more looks, fewer top seeds), other phase windows (the present ones are within 5% of the optimum for this card), more threads on CPU looks or more workers than the settings in use (flat), the 1,536-block window as a score term (section 5), the other kernels built for the default architecture (terrain a third slower, the rest the same).

That the gate, the scan and the workers each sat on a flat optimum was true of the settings and not of the code: the last rental changed no setting that mattered and raised the gate's output by half. What the tuning shows is where the next gain is not. The card is busy (of the gate's share: 68% scan, 16% coarse look, 10% first look), and what would free more of it is the workers' side: their terrain, table and density requests hold four tenths of the card and were written quickly and never tuned.

An aside the data answered: none of the big caverns is free of deep dark (the top 20 have it in 14 to 52% of their air), because deep dark sits under exactly the mountain terrain that giants need. The largest deep-dark-free cavern among the measured ones is 17,724,200 blocks, seed 1382579, `/tp 26881896 1 -12313227`.

7. The top of the list

#	blocks	seed	/tp	size x, y, z
1	55,830,955	6008414183811625950	19079170 13 -7883600	2042 × 238 × 2130
2	55,213,072	-5741848770048959941	24341428 18 -20812181	2671 × 202 × 1769
3	54,581,532	208969807933184	-14756350 24 -17860759	1505 × 243 × 2173
4	54,531,910	263674329346503	-22888218 15 5435869	2082 × 249 × 2018
5	53,104,476	3105282197641714333	4681318 27 27823818	1740 × 271 × 1816
6	52,779,093	7161509186883105506	-1309621 22 -20297734	1869 × 263 × 2213
7	51,782,360	270257040192206	-16538195 18 -24116083	2399 × 238 × 1779
8	51,696,456	378236675125408	-4707445 38 -28140970	1579 × 257 × 2099
9	51,606,423	495631510238219	-5789962 18 -11603402	1968 × 224 × 1573
10	51,504,851	674655766032442	21835040 21 -21694392	1546 × 239 × 2189
11	51,460,023	4507554610487569579	-3256953 22 -28278412	1549 × 220 × 1956
12	50,992,456	78389607299707	22371198 20 -9706918	1620 × 250 × 1977
13	50,647,905	-2807124812770822876	6034390 28 -18457492	1471 × 273 × 2148
14	50,641,791	-375122011334095778	3120602 33 -8870210	1626 × 260 × 1981

8. Files

`gpuscan.cu`, `gpulook2.cu`, `gputables.cu`, `seedgate14.cpp` (the gate), `cpuprelook2.hpp` (the coarse looks), `gpusrv.cpp` with `gputerrain.cu`, `gpupairs.cu`, `gpuformula.cu`, `native/gpudensity.cu` (the GPU's services for the workers), `cavepairs5.cpp`, `formulacave5.cpp` (the scanner and formula stage that use them), `analyticgate.hpp` (the closed-form window means), `percgate.hpp` (the percolation term), `stats/alignplaces.cpp` (the complete alignment search), `cavepairs3.cpp` (the scanner), `formulacave.cpp`, `native/cavemeasure.cpp` and `native/gamegen.hpp` (the exact measure), `search3.py` (the driver), `measure.sh` (the game's own code), `stats/` (fits and probes), `hits/` (every exact measurement), `stats/sitefeatures.cpp`, `stats/layerfeatures.cpp`, `stats/gatefeatures3.cpp` and `stats/limits/` (section 5), `fleet/setup5.sh` and `fleet/box/start_g14.sh` (a box from nothing), `docs/plots.py` (these figures).